MongoDB Practical - students Collection (instituteDB)

1.

```
use('instituteDB');
```

The use('instituteDB'); command is used to select or switch to the instituteDB database in MongoDB.

Output:

```
switched to db instituteDB
```

2.

```
db.students.insertOne({
name: "Arjun",
age: 21,
marks: 68,
course: "BBA",
city: "Nagpur"
});
```

db.collection.insertOne(document) is the general syntax used to insert a single document into a MongoDB collection.

Output:

```
{
acknowledged: true,
insertedId: ObjectId('6ba4c123b1612dd272d10a1c')
}
```

3.

```
db.students.insertMany([
{
name: "Meera",
age: 19,
marks: 78,
course: "BBA",
city: "Nagpur"
},
{
name: "Kabir",
age: 22,
marks: 91,
course: "BTech",
city: "Delhi"
},
{
name: "Divya",
age: 20,
marks: 38,
course: "BA",
city: "Jaipur"
},
{
name: "Tanvi",
age: 21,
marks: 84,
course: "BTech",
city: "Delhi"
},
{
name: "Aditya",
age: 23,
marks: 29,
course: "BA",
city: "Nagpur"
},
{
name: "Yash",
age: 19,
marks: 34,
```

```
course: "BA",
city: "Jaipur"
},
{
name: "Manish",
age: 22,
marks: 62,
course: "BA",
city: "Indore"
},
{
name: "Ishita",
age: 20,
marks: 95,
course: "BTech",
city: "Indore"
}
]);
```

db.collection.insertMany([document1, document2, ...]) is used to insert multiple documents into a MongoDB collection at the same time.

Output:

```
{
acknowledged: true,
insertedIds: {
'0': ObjectId('6b371c17149d439536b30000'),
'1': ObjectId('6b371c17149d439536b30001'),
'2': ObjectId('6b371c17149d439536b30002'),
'3': ObjectId('6b371c17149d439536b30003'),
'4': ObjectId('6b371c17149d439536b30004'),
'5': ObjectId('6b371c17149d439536b30005'),
'6': ObjectId('6b371c17149d439536b30006'),
'7': ObjectId('6b371c17149d439536b30007')
}
}
```

4.

```
db.students.find();
```

db.collection.find() is used to display all documents/records stored in a MongoDB collection.

Output:

```
{
 _id: ObjectId('6ba4c123b1612dd272d10a1c'),
 name: 'Arjun',
 age: NumberInt('21'),
 marks: NumberInt('68'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('78'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30001'),
 name: 'Kabir',
 age: NumberInt('22'),
 marks: NumberInt('91'),
 course: 'BTech',
 city: 'Delhi'
}
{
 _id: ObjectId('6b371c17149d439536b30002'),
 name: 'Divya',
 age: NumberInt('20'),
```

```
 marks: NumberInt('38'),
 course: 'BA',
 city: 'Jaipur'
}
{
 _id: ObjectId('6b371c17149d439536b30003'),
 name: 'Tanvi',
 age: NumberInt('21'),
 marks: NumberInt('84'),
 course: 'BTech',
 city: 'Delhi'
}
{
 _id: ObjectId('6b371c17149d439536b30004'),
 name: 'Aditya',
 age: NumberInt('23'),
 marks: NumberInt('29'),
 course: 'BA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30005'),
 name: 'Yash',
 age: NumberInt('19'),
 marks: NumberInt('34'),
 course: 'BA',
 city: 'Jaipur'
}
{
 _id: ObjectId('6b371c17149d439536b30006'),
 name: 'Manish',
 age: NumberInt('22'),
 marks: NumberInt('62'),
 course: 'BA',
 city: 'Indore'
}
{
 _id: ObjectId('6b371c17149d439536b30007'),
 name: 'Ishita',
 age: NumberInt('20'),
 marks: NumberInt('95'),
 course: 'BTech',
 city: 'Indore'
}
```

5.

```
db.students.find({
marks: { $gt: 90 }
});
```

This command searches the students collection and displays only those students whose marks are greater than 90.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30001'),
 name: 'Kabir',
 age: NumberInt('22'),
 marks: NumberInt('91'),
 course: 'BTech',
 city: 'Delhi'
}
{
 _id: ObjectId('6b371c17149d439536b30007'),
 name: 'Ishita',
 age: NumberInt('20'),
 marks: NumberInt('95'),
 course: 'BTech',
 city: 'Indore'
}
```

6.

```
db.students.find({
age: { $gte: 22 }
});
```

This command searches the students collection and displays all students whose age is 22 or greater.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30001'),
 name: 'Kabir',
 age: NumberInt('22'),
 marks: NumberInt('91'),
 course: 'BTech',
 city: 'Delhi'
}
{
 _id: ObjectId('6b371c17149d439536b30004'),
 name: 'Aditya',
 age: NumberInt('23'),
 marks: NumberInt('29'),
 course: 'BA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30006'),
 name: 'Manish',
 age: NumberInt('22'),
 marks: NumberInt('62'),
 course: 'BA',
 city: 'Indore'
}
```

7.

```
db.students.find({
marks: { $lte: 40 }
});
```

This command searches the students collection and displays all students whose marks are 40 or less.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30002'),
 name: 'Divya',
 age: NumberInt('20'),
 marks: NumberInt('38'),
 course: 'BA',
 city: 'Jaipur'
}
{
 _id: ObjectId('6b371c17149d439536b30004'),
 name: 'Aditya',
 age: NumberInt('23'),
 marks: NumberInt('29'),
 course: 'BA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30005'),
 name: 'Yash',
 age: NumberInt('19'),
 marks: NumberInt('34'),
 course: 'BA',
 city: 'Jaipur'
}
```

8.

```
db.students.find({
age: { $lte: 20 }
});
```

This command searches the students collection and displays all students whose age is 20 or less.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('78'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30002'),
 name: 'Divya',
 age: NumberInt('20'),
 marks: NumberInt('38'),
 course: 'BA',
 city: 'Jaipur'
}
{
 _id: ObjectId('6b371c17149d439536b30005'),
 name: 'Yash',
 age: NumberInt('19'),
 marks: NumberInt('34'),
 course: 'BA',
 city: 'Jaipur'
}
{
 _id: ObjectId('6b371c17149d439536b30007'),
 name: 'Ishita',
 age: NumberInt('20'),
 marks: NumberInt('95'),
 course: 'BTech',
 city: 'Indore'
}
```

9.

```
db.students.find({
marks: 78
});
```

This command searches the students collection and displays all students whose marks are exactly 78.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('78'),
 course: 'BBA',
 city: 'Nagpur'
}
```

10.

```
db.students.find({
marks: { $gt: 50 },
city: "Nagpur"
});
```

This command searches the students collection and displays students who satisfy both conditions:

- Their marks are greater than 50

- Their city is Nagpur

MongoDB treats the two conditions together as AND.

Output:

```
{
 _id: ObjectId('6ba4c123b1612dd272d10a1c'),
 name: 'Arjun',
 age: NumberInt('21'),
```

```
 marks: NumberInt('68'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('78'),
 course: 'BBA',
 city: 'Nagpur'
}
```

11.

```
db.students.deleteOne({
name: "Aditya"
});
```

This command searches the students collection for a student named Aditya and deletes that student's document.

Output:

```
{
acknowledged: true,
deletedCount: NumberInt('1')
}
```

12.

```
db.students.deleteOne({
marks: { $lt: 40 }
});
```

This command finds one student whose marks are less than 40 (the first match found) and deletes that student's document. Here, Divya is deleted.

Output:

```
{
acknowledged: true,
deletedCount: NumberInt('1')
}
```

13.

```
db.students.deleteMany({
marks: { $lt: 40 }
});
```

This command finds and deletes all students whose marks are less than 40 from the students collection. Only Yash is still below 40, so one document is deleted.

Output:

```
{
acknowledged: true,
deletedCount: NumberInt('1')
}
```

14.

```
db.students.updateOne(
{ name: "Meera" },
{ $set: { marks: 85 } }
);
```

This command finds Meera's document in the students collection and changes her marks to 85. It updates only one matching document.

Output:

```
{
acknowledged: true,
insertedId: null,
matchedCount: NumberInt('1'),
modifiedCount: NumberInt('1'),
upsertedCount: NumberInt('0')
}
```

15.

```
db.students.find({
name: "Meera"
});
```

This command searches the students collection and displays the record of the student whose name is Meera.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('85'),
 course: 'BBA',
 city: 'Nagpur'
}
```

16.

```
db.students.updateOne(
{ marks: { $lt: 40 } },
{ $set: { marks: 40 } }
);
```

This command finds one student whose marks are less than 40 and changes their marks to 40. Here, $lt means less than, and $set is used to update the value. No student has marks below 40 now, so nothing is matched.

Output:

```
{
acknowledged: true,
insertedId: null,
matchedCount: NumberInt('0'),
modifiedCount: NumberInt('0'),
upsertedCount: NumberInt('0')
}
```

17.

```
db.students.updateMany(
{ marks: { $gt: 70 } },
{ $inc: { marks: 5 } }
);
```

This command finds all students whose marks are greater than 70 and increases their marks by 5. Here, $gt means greater than, and $inc is used to increase a numeric value by a specified amount.

Output:

```
{
acknowledged: true,
insertedId: null,
matchedCount: NumberInt('4'),
modifiedCount: NumberInt('4'),
upsertedCount: NumberInt('0')
}
```

18.

```
db.students.find().limit(2);
```

This command displays only the first 2 documents from the students collection.

Output:

```
{
 _id: ObjectId('6ba4c123b1612dd272d10a1c'),
 name: 'Arjun',
 age: NumberInt('21'),
 marks: NumberInt('68'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
```

```
 marks: NumberInt('90'),
 course: 'BBA',
 city: 'Nagpur'
}
```

19.

```
db.students.find({
marks: { $gt: 40 }
}).limit(2);
```

This command finds students whose marks are greater than 40 and displays only the first 2 matching students.

Output:

```
{
 _id: ObjectId('6ba4c123b1612dd272d10a1c'),
 name: 'Arjun',
 age: NumberInt('21'),
 marks: NumberInt('68'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('90'),
 course: 'BBA',
 city: 'Nagpur'
}
```

20.

```
db.students.find().sort({
marks: 1
});
```

This command displays all students from the students collection and arranges them according to their marks in ascending order.

Output:

```
{
 _id: ObjectId('6b371c17149d439536b30006'),
 name: 'Manish',
 age: NumberInt('22'),
 marks: NumberInt('62'),
 course: 'BA',
 city: 'Indore'
}
{
 _id: ObjectId('6ba4c123b1612dd272d10a1c'),
 name: 'Arjun',
 age: NumberInt('21'),
 marks: NumberInt('68'),
 course: 'BBA',
 city: 'Nagpur'
}
{
 _id: ObjectId('6b371c17149d439536b30003'),
 name: 'Tanvi',
 age: NumberInt('21'),
 marks: NumberInt('89'),
 course: 'BTech',
 city: 'Delhi'
}
{
 _id: ObjectId('6b371c17149d439536b30000'),
 name: 'Meera',
 age: NumberInt('19'),
 marks: NumberInt('90'),
 course: 'BBA',
 city: 'Nagpur'
```

```
}
{
 _id: ObjectId('6b371c17149d439536b30001'),
 name: 'Kabir',
 age: NumberInt('22'),
 marks: NumberInt('96'),
 course: 'BTech',
 city: 'Delhi'
}
{
 _id: ObjectId('6b371c17149d439536b30007'),
 name: 'Ishita',
 age: NumberInt('20'),
 marks: NumberInt('100'),
 course: 'BTech',
 city: 'Indore'
}
```

21.

```
db.students.deleteMany({});
```

This command removes all documents from the students collection so the data can be loaded again from scratch.

Output:

```
{
acknowledged: true,
deletedCount: NumberInt('6')
}
```

22.

```
db.students.insertMany([
{
name: "Meera",
age: 19,
marks: 78,
course: "BBA",
city: "Nagpur"
},
{
name: "Kabir",
age: 22,
marks: 91,
course: "BTech",
city: "Delhi"
},
{
name: "Divya",
age: 20,
marks: 38,
course: "BA",
city: "Jaipur"
},
{
name: "Tanvi",
age: 21,
marks: 84,
course: "BTech",
city: "Delhi"
},
{
name: "Aditya",
age: 23,
marks: 29,
course: "BA",
city: "Nagpur"
},
{
name: "Yash",
age: 19,
marks: 34,
```

```
course: "BA",
city: "Jaipur"
},
{
name: "Manish",
age: 22,
marks: 62,
course: "BA",
city: "Indore"
},
{
name: "Ishita",
age: 20,
marks: 95,
course: "BTech",
city: "Indore"
}
]);
```

The same eight documents are inserted again into the now-empty collection.

Output:

```
{
acknowledged: true,
insertedIds: {
'0': ObjectId('6b216fdaeeb975729fae0000'),
'1': ObjectId('6b216fdaeeb975729fae0001'),
'2': ObjectId('6b216fdaeeb975729fae0002'),
'3': ObjectId('6b216fdaeeb975729fae0003'),
'4': ObjectId('6b216fdaeeb975729fae0004'),
'5': ObjectId('6b216fdaeeb975729fae0005'),
'6': ObjectId('6b216fdaeeb975729fae0006'),
'7': ObjectId('6b216fdaeeb975729fae0007')
}
}
```

23.

```
db.students.find({}).sort({ marks: 1 });
```

This command retrieves all students from the students collection and sorts them according to their marks in ascending order.

Output:

```
[
 {
 name: "Aditya",
 age: 23,
 marks: 29,
 course: "BA",
 city: "Nagpur"
 },
 {
 name: "Yash",
 age: 19,
 marks: 34,
 course: "BA",
 city: "Jaipur"
 },
 {
 name: "Divya",
 age: 20,
 marks: 38,
 course: "BA",
 city: "Jaipur"
 },
 {
 name: "Manish",
 age: 22,
 marks: 62,
 course: "BA",
 city: "Indore"
```

```
},
{
name: "Meera",
age: 19,
marks: 78,
course: "BBA",
city: "Nagpur"
},
{
name: "Tanvi",
age: 21,
marks: 84,
course: "BTech",
city: "Delhi"
},
{
name: "Kabir",
age: 22,
marks: 91,
course: "BTech",
city: "Delhi"
},
{
name: "Ishita",
age: 20,
marks: 95,
course: "BTech",
city: "Indore"
}
]
```

24.

```
db.students.find().sort({ age: 1 });
```

This command displays all students from the students collection and arranges them according to their age in ascending order.

Output:

```
[
{
name: "Meera",
age: 19,
marks: 78,
course: "BBA",
city: "Nagpur"
},
{
name: "Yash",
age: 19,
marks: 34,
course: "BA",
city: "Jaipur"
},
{
name: "Divya",
age: 20,
marks: 38,
course: "BA",
city: "Jaipur"
},
{
name: "Ishita",
age: 20,
marks: 95,
course: "BTech",
city: "Indore"
},
{
name: "Tanvi",
age: 21,
```

```
 marks: 84,
 course: "BTech",
 city: "Delhi"
 },
 {
 name: "Kabir",
 age: 22,
 marks: 91,
 course: "BTech",
 city: "Delhi"
 },
 {
 name: "Manish",
 age: 22,
 marks: 62,
 course: "BA",
 city: "Indore"
 },
 {
 name: "Aditya",
 age: 23,
 marks: 29,
 course: "BA",
 city: "Nagpur"
 }
]
```

25.

```
db.students.find().sort({ age: -1 });
```

This command displays all students from the students collection and arranges them according to their age in descending order.

Output:

```
[
 {
 name: "Aditya",
 age: 23,
 marks: 29,
 course: "BA",
 city: "Nagpur"
 },
 {
 name: "Kabir",
 age: 22,
 marks: 91,
 course: "BTech",
 city: "Delhi"
 },
 {
 name: "Manish",
 age: 22,
 marks: 62,
 course: "BA",
 city: "Indore"
 },
 {
 name: "Tanvi",
 age: 21,
 marks: 84,
 course: "BTech",
 city: "Delhi"
 },
 {
 name: "Divya",
 age: 20,
 marks: 38,
 course: "BA",
 city: "Jaipur"
 },
```

```
 {
 name: "Ishita",
 age: 20,
 marks: 95,
 course: "BTech",
 city: "Indore"
 },
 {
 name: "Meera",
 age: 19,
 marks: 78,
 course: "BBA",
 city: "Nagpur"
 },
 {
 name: "Yash",
 age: 19,
 marks: 34,
 course: "BA",
 city: "Jaipur"
 }
]
```

26.

```
db.students.find()
.sort({ marks: -1 })
.limit(3);
```

This command displays the top 3 students based on their marks. First, it sorts all students by marks in descending order (highest to lowest), and then limit(3) displays only the first 3 students.

Output:

```
[
 {
 name: "Ishita",
 age: 20,
 marks: 95,
 course: "BTech",
 city: "Indore"
 },
 {
 name: "Kabir",
 age: 22,
 marks: 91,
 course: "BTech",
 city: "Delhi"
 },
 {
 name: "Tanvi",
 age: 21,
 marks: 84,
 course: "BTech",
 city: "Delhi"
 }
]
```

27.

```
db.students.find().sort({ marks: -1 }).limit(1);
```

This command finds the student with the highest marks in the students collection. It first sorts all students' marks in descending order and then limit(1) displays only the first student.

Output:

```
{
 name: "Ishita",
 age: 20,
 marks: 95,
 course: "BTech",
 city: "Indore"
 }
```

28.

```
db.students.find(
{},
{
_id: 0,
name: 1,
marks: 1
}
).sort({ marks: -1 }).limit(1);
```

This command finds the student with the highest marks and displays only their name and marks.

Output:

```
{
 name: "Ishita",
 marks: 95
}
```